TECNOLÓGICO DE ESTUDIOS SUPERIORES DE IXTAPALUCA

INTEGRANTES:

GUZMÁN LÓPEZ JESÚS

LOPEZ MORALES ELIZABETH

GONZALEZ MENDOZA CARLOS FAUSTINO

RAMÍREZ MONTERRUBIO JULIO CESAR

PROFESOR:

ING. ESTEBAN RAMÍREZ DE LA ROSA.

PROYECTO:

REDES NEURONALES ARTIFICIALES (UNIDADES 2 Y 3)
DEL PERCEPTRÓN A LA RETROPROPAGACIÓN

GRUPO:1951-1952 **SEMESTRE:** 9°

FECHA DE ENTREGA: OCTUBRE DEL 2026

Implementación de una red neuronal multicapa para resolver la compuerta XOR

Objetivo y descripción del problema

El objetivo de esta práctica es implementar y entrenar una red neuronal multicapa mediante el algoritmo de retropropagación para resolver la compuerta lógica XOR. Asimismo, se compara el efecto de dos tasas de aprendizaje sobre la reducción del error durante el entrenamiento.

La compuerta XOR produce una salida de 1 cuando sus entradas son diferentes y una salida de 0 cuando son iguales. Este problema no es linealmente separable, por lo que requiere una red con al menos una capa oculta y funciones de activación no lineales.

Entrada 1	Entrada 2	Salida esperada
0	0	0
0	1	1
1	0	1
1	1	0

1. Diseño de la arquitectura

Se implementó una red neuronal multicapa para clasificar las cuatro combinaciones de la compuerta lógica XOR. Su arquitectura está formada por una capa de entrada, una capa oculta y una capa de salida, con una distribución de **2–4–1 neuronas**.

Capa	Neuronas	Activación	Justificación
Entrada	2	Sin activación propia	Recibe los dos valores binarios de XOR.
Oculta	4	Tangente hiperbólica (tanh)	Introduce la no linealidad necesaria para aprender XOR. Cuatro neuronas constituyen una arquitectura pequeña que resultó suficiente en los experimentos.
Salida	1	Sigmoide	Genera una salida entre 0 y 1, adecuada para una clasificación binaria.

Ilustración 1 Arquitectura de la red neuronal multicapa para XOR

La capa oculta es necesaria porque XOR no es linealmente separable: sus clases no pueden distinguirse mediante una sola recta. La función tanh permite transformar las entradas y construir una separación no lineal.

Para interpretar la salida, se utilizó un umbral de **0.5**: los valores mayores o iguales se clasifican como 1 y los menores como 0. La implementación se desarrolló en Python con NumPy para los cálculos y Matplotlib para las gráficas.

2. Entrenamiento

El entrenamiento se realizó utilizando los cuatro casos de la tabla de verdad de XOR y sus salidas esperadas: **0, 1, 1 y 0**. En cada época se procesaron conjuntamente los cuatro ejemplos y se realizaron las siguientes etapas:

- **Propagación hacia adelante —Forward Pass—.** Las entradas se multiplicaron por los pesos de la primera capa y se sumaron los sesgos. Después se aplicó tanh. Las salidas de la capa oculta se procesaron con los pesos y sesgo de salida, aplicando finalmente la función sigmoide:

$$H = \tanh(XW_1 + b_1)$$

$$\hat{Y} = \text{sigmoide}(HW_2 + b_2)$$

- **Cálculo del error.** Las predicciones se compararon con las respuestas esperadas mediante la entropía cruzada binaria:

$$L = -\frac{1}{4}\sum_{i=1}^{4}[y_i \ln(\hat{y}_i) + (1 - y_i)\ln(1 - \hat{y}_i)]$$

Una pérdida menor indica que las predicciones se acercan a las respuestas esperadas.

- **Retropropagación —Backpropagation—.** Se calcularon los gradientes de la pérdida respecto a los pesos y sesgos, comenzando por la capa de salida y continuando hacia la capa oculta. Para ello se utilizaron la regla de la cadena y las derivadas de las funciones de activación.
- **Actualización de pesos.** Los pesos y sesgos se ajustaron mediante descenso de gradiente:

$$W_{\text{nuevo}} = W_{\text{actual}} - \eta \frac{\partial L}{\partial W}$$

Aquí, η representa la tasa de aprendizaje. El procedimiento se repitió durante 5,000 épocas en cada experimento.

3. Experimentación

Se realizaron dos experimentos para comparar el efecto de la tasa de aprendizaje. Se mantuvieron constantes la arquitectura, los datos, el número de épocas y la semilla aleatoria. La semilla

Parámetro	Experimento 1	Experimento 2
Arquitectura	2–4–1	2–4–1
Tasa de aprendizaje	0.1	1.0
Épocas	5,000	5,000
Semilla	42	42
Pérdida final	0.004109	0.000234
Exactitud sobre los cuatro casos	100 %	100 %

Ilustración 2 Configuración y resultados de los experimentos de entrenamiento.

42 permitió que ambos entrenamientos comenzaran con los mismos pesos iniciales.

Entrada	Esperado	Salida con tasa 0.1	Salida con tasa 1.0	Clase en ambos
[0, 0]	0	0.001272	0.000093	0
[0, 1]	1	0.995232	0.999729	1
[1, 0]	1	0.995732	0.999755	1
[1, 1]	0	0.006087	0.000327	0

Ilustración 3 Comparación de las predicciones de XOR con tasas de aprendizaje de 0.1 y 1.0.

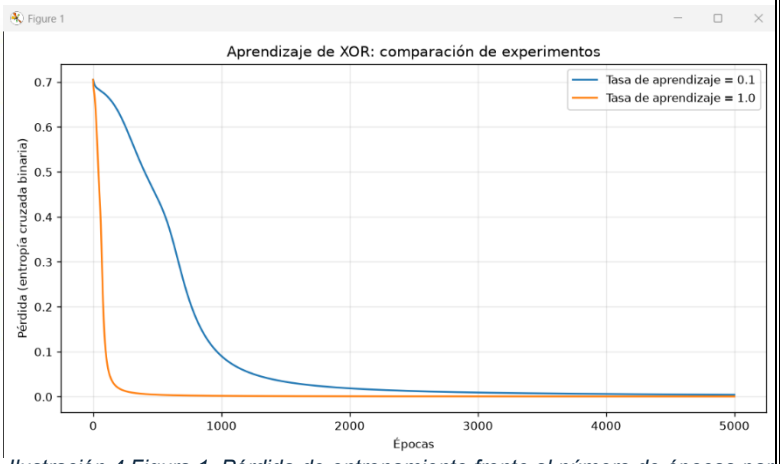


Ilustración 4 Figura 1. Pérdida de entrenamiento frente al número de épocas para las tasas de aprendizaje 0.1 y 1.0.

Análisis de los resultados

Convergencia. En ambos experimentos, la pérdida disminuyó y se aproximó a cero. Las dos configuraciones lograron clasificar correctamente los cuatro casos de XOR. Sin embargo, la curva de la tasa 1.0 descendió más rápidamente y terminó con una pérdida de 0.000234, mientras que la tasa 0.1 obtuvo una pérdida de 0.004109. Aunque ambos experimentos alcanzaron el 100 % de exactitud, sus pérdidas fueron diferentes porque la pérdida también considera los valores numéricos de las predicciones. Con la tasa 1.0, las salidas quedaron más próximas a los valores esperados de 0 y 1.

Sobreajuste. No se puede afirmar que hubo o que no hubo sobreajuste a partir de esta evaluación. Se utilizaron los mismos cuatro casos para entrenar y comprobar las respuestas, sin un conjunto de validación independiente. Además, esos casos representan todas las combinaciones binarias de XOR. Por ello, los resultados demuestran que la red aprendió la tabla de verdad, pero no permiten evaluar su generalización a otros tipos de entradas.

Efecto de la tasa de aprendizaje. La tasa 0.1 produjo ajustes más pequeños y una reducción del error más lenta. La tasa 1.0 permitió ajustes mayores y una convergencia más rápida, sin oscilaciones visibles en la gráfica obtenida. Esta ventaja se observó bajo las condiciones de esta ejecución; no significa que aumentar la tasa siempre mejore el entrenamiento.

Conclusión

La red neuronal multicapa de arquitectura 2–4–1 aprendió correctamente los cuatro casos de XOR mediante retropropagación. Ambos experimentos alcanzaron el 100 % de exactitud, pero la tasa de aprendizaje 1.0 redujo el error más rápidamente y obtuvo una menor pérdida final. La práctica permitió comprobar la importancia de la capa oculta, las activaciones no lineales y la elección de la tasa de aprendizaje durante el entrenamiento.